Darshan UNIVERSITY

योग: कर्मसु कौशलम्

Python Programming - 2301CS404

Lab - 15 (2)

Roll No.: 418

Name: Bhavik A. Parmar

Continued..

10) Calculate area of a ractangle using object as an argument to a method.

In [1]:
```
class Rectangle:

    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self, obj):
        return obj.length * obj.breadth

def main():
    r1 = Rectangle(5, 3)

    result = r1.area(r1)

    print("Area of Rectangle:", result)

main()
```

```
Area of Rectangle: 15
```

11) Calculate the area of a square.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

In [2]:
```
class Square:

    def __init__(self, side):
        self.side = side

    def area(self):
        result = self.side * self.side
        self.output(result)   # Calling output() method

    def output(self, area):
        print("Side of Square:", self.side)
        print("Area of Square:", area)


def main():
    s = float(input("Enter side of square: "))

    sq = Square(s)
    sq.area()

main()
```

```
Side of Square: 4.0
Area of Square: 16.0
```

12) Calculate the area of a rectangle.

Include a Constructor, a method to calculate area named area() and a method named output() that prints the output and is invoked by area().

Also define a class method that compares the two sides of reactangle. An object is instantiated only if the two sides are different; otherwise a message should be displayed : THIS IS SQUARE.

In [3]:
```
class Rectangle:

    def __init__(self, length, breadth):
        self.length = length
        self.breadth = breadth

    def area(self):
        result = self.length * self.breadth
        self.output(result)   # calling output()

    def output(self, area):
        print("\nLength:", self.length)
        print("Breadth:", self.breadth)
        print("Area of Rectangle:", area)

    @classmethod
    def check_shape(cls, length, breadth):
        if length == breadth:
            print("THIS IS SQUARE")
            return None
        else:
```

```
            return cls(length, breadth)

def main():
    l = float(input("Enter length: "))
    b = float(input("Enter breadth: "))

    rect = Rectangle.check_shape(l, b)

    if rect:
        rect.area()

main()
```

```
Length: 5.0
Breadth: 3.0
Area of Rectangle: 15.0
```

13) Define a class Square having a private attribute "side".

Implement get_side and set_side methods to accees the private attribute from outside of the class.

In [4]:
```
class Square:

    def __init__(self, side):
        self.__side = side   # private attribute

    def set_side(self, side):
        self.__side = side

    def get_side(self):
        return self.__side


def main():
    sq = Square(5)

    print("Initial Side:", sq.get_side())

    sq.set_side(10)

    print("Updated Side:", sq.get_side())

main()
```

```
Initial Side: 5
Updated Side: 10
```

14) Create a class Profit that has a method named getProfit that accepts profit from the user.

Create a class Loss that has a method named getLoss that accepts loss from the user.

Create a class BalanceSheet that inherits from both classes Profit and Loss and calculates the balanace. It has two

methods getBalance() and printBalance().

In [5]:
```
class Profit:

    def getProfit(self):
        self.profit = float(input("Enter Profit: "))

class Loss:

    def getLoss(self):
        self.loss = float(input("Enter Loss: "))


class BalanceSheet(Profit, Loss):

    def getBalance(self):
        self.balance = self.profit - self.loss

    def printBalance(self):
        print("\n--- Balance Sheet ---")
        print("Profit:", self.profit)
        print("Loss:", self.loss)
        print("Balance:", self.balance)


def main():
    bs = BalanceSheet()

    bs.getProfit()
    bs.getLoss()

    bs.getBalance()
    bs.printBalance()

main()
```

```
--- Balance Sheet ---
Profit: 5000.0
Loss: 2000.0
Balance: 3000.0
```

15) WAP to demonstrate all types of inheritance.

In [6]:
```
#  🔹 1. Single Inheritance
class A:
    def showA(self):
        print("Class A")

class B(A):
    def showB(self):
        print("Class B (Single Inheritance)")


#  🔹 2. Multiple Inheritance
class C:
    def showC(self):
        print("Class C")

class D(A, C):
```

```
    def showD(self):
        print("Class D (Multiple Inheritance)")


#  🔹 3. Multilevel Inheritance
class E(B):
    def showE(self):
        print("Class E (Multilevel Inheritance)")


#  🔹 4. Hierarchical Inheritance
class F(A):
    def showF(self):
        print("Class F (Hierarchical Inheritance)")


#  🔹 5. Hybrid Inheritance (Combination)
class G(D, F):
    def showG(self):
        print("Class G (Hybrid Inheritance)")


#  🔹 Main Method
def main():
    print("\n--- Single Inheritance ---")
    b = B()
    b.showA()
    b.showB()

    print("\n--- Multiple Inheritance ---")
    d = D()
    d.showA()
    d.showC()
    d.showD()

    print("\n--- Multilevel Inheritance ---")
    e = E()
    e.showA()
    e.showB()
    e.showE()

    print("\n--- Hierarchical Inheritance ---")
    f = F()
    f.showA()
    f.showF()

    print("\n--- Hybrid Inheritance ---")
    g = G()
    g.showA()
    g.showC()
    g.showD()
    g.showF()
    g.showG()


# Run program
main()
```

```
--- Single Inheritance ---
Class A
Class B (Single Inheritance)

--- Multiple Inheritance ---
Class A
Class C
Class D (Multiple Inheritance)

--- Multilevel Inheritance ---
Class A
Class B (Single Inheritance)
Class E (Multilevel Inheritance)

--- Hierarchical Inheritance ---
Class A
Class F (Hierarchical Inheritance)

--- Hybrid Inheritance ---
Class A
Class C
Class D (Multiple Inheritance)
Class F (Hierarchical Inheritance)
Class G (Hybrid Inheritance)
```

16) Create a Person class with a constructor that takes two arguments name and age.

Create a child class Employee that inherits from Person and adds a new attribute salary.

Override the **init** method in Employee to call the parent class's **init** method using the super() and then initialize the salary attribute.

In [7]:
```python
class Person:

    def __init__(self, name, age):
        self.name = name
        self.age = age

class Employee(Person):

    def __init__(self, name, age, salary):
        super().__init__(name, age)   # Call parent constructor
        self.salary = salary

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Salary:", self.salary)


def main():
    emp = Employee("Bhavik", 18, 30000)
    emp.display()
```

```
main()
```

```
Name: Bhavik
Age: 18
Salary: 30000
```

17) Create a Shape class with a draw method that is not implemented.

Create three child classes Rectangle, Circle, and Triangle that implement the draw method with their respective drawing behaviors.

Create a list of Shape objects that includes one instance of each child class, and then iterate through the list and call the draw method on each object.

In [8]:
```
# Base Class
class Shape:

    # Method not implemented
    def draw(self):
        pass


# Child Class: Rectangle
class Rectangle(Shape):
    def draw(self):
        print("Drawing Rectangle")


# Child Class: Circle
class Circle(Shape):
    def draw(self):
        print("Drawing Circle")


# Child Class: Triangle
class Triangle(Shape):
    def draw(self):
        print("Drawing Triangle")


#  🔹 Main Method
def main():
    # List of Shape objects
    shapes = [Rectangle(), Circle(), Triangle()]

    # Iterate and call draw()
    for shape in shapes:
        shape.draw()


# Run program
main()
```

```
Drawing Rectangle
Drawing Circle
Drawing Triangle
```